Cab Booking App Project Overview

Cab Booking App Project Overview

The Uber clone project is a **full-stack ride-hailing application** that allows users to book rides, drivers to accept requests, and payments to be processed seamlessly. This project will be built using:

- **Next.js & React.js** – For a fast and scalable frontend.
- **Tailwind CSS** – For modern and responsive UI styling.
- **Clerk, Supabase, Apprite** – For secure user authentication (sign-up, login, social auth).
- **Stripe** – For handling ride payments.
- **Backend** - Express js and Node js
- **Database** - Supabase, Firebase, PostgresSQL

Project Features:

✅ **User Authentication:** Riders and drivers can sign up, log in, and manage profiles using Clerk.
✅ **Ride Booking System:** Users can enter pickup and drop-off locations and request a ride.
✅ **Driver Dashboard:** Drivers can accept or decline ride requests.
✅ **Google Maps Integration:** Displays routes, estimated fares, and driver tracking.
✅ **Real-Time Updates:** Rides update dynamically using WebSockets or Firebase.
✅ **Payment Gateway (Stripe):** Secure payment processing for riders and earnings for drivers.
✅ **Ride History & Receipts:** Users can view past rides, and payments and download receipts.
✅ **Ratings & Reviews:** Users can rate drivers and leave feedback.
✅ **Mobile Responsive UI:** The app is optimized for mobile and desktop screens.(Optional)

Benefits of Completing This Project for Students

Building an Uber clone will provide students with **real-world development experience** and enhance their technical and problem-solving skills. Here's how:

🚀 1. Full-Stack Development Mastery

- Hands-on experience with **Next.js & React.js** for frontend development.
- Learning how to build and secure APIs with **Next.js API routes or Node.js**.
- Database handling with **PostgreSQL, Firebase, or Supabase**.

💡 2. Practical Experience with Authentication & Security

- Implement secure authentication and role-based access using **Clerk**.
- Learn about **OAuth, JWT, and session-based authentication**.

💰 3. Understanding Payment Gateways

- Gain experience in integrating **Stripe for payments**, an in-demand skill for e-commerce and SaaS applications.
- Learn **handling transactions, webhooks, and payment security**.

📍 4. Mastering Real-Time Features & WebSockets

- Implement **real-time ride tracking, notifications, and ride status updates**.
- Learn to work with **WebSockets, Firebase, or Pusher** for real-time communication.

🎨 5. UI/UX Design & Responsive Development

- Hands-on experience with **Tailwind CSS** for building a sleek, modern UI.

- Learning to make applications **mobile-friendly** and **accessible**.

📡 6. Working with Third-Party APIs

- Get comfortable integrating **Google Maps API** for ride tracking and directions.
- Learn how to fetch and display real-time data from external sources.

🚀 7. Deploying a Scalable Web App

- Experience deploying full-stack applications using **Vercel, Netlify, or a backend hosting service**.
- Learn about **CI/CD (Continuous Integration/Continuous Deployment)** to automate app updates.

💼 8. Job Readiness & Portfolio Building

By completing this project, students can:
✅ **Showcase a professional-level full-stack project on their portfolio.**
✅ **Stand out in job interviews by demonstrating practical, hands-on skills.**
✅ **Confidently apply for roles such as Frontend Developer, Full-Stack Developer, or Software Engineer.**

Conclusion

This project is a **perfect blend of frontend, backend, real-time features, and payment integration**, making it an **excellent learning experience** for students looking to boost their **technical skills, confidence, and employability**.

🔥 After completing this, students can **work on freelancing gigs, create their own startups, or apply for high-paying developer jobs**! 🚀

Here's a two-week schedule for building your Uber clone:

Week 1: Setting Up & Core Features

Day 1: Project Setup & Authentication

- Initialize the Next.js project.
- Install and configure Tailwind CSS.
- Set up Clerk for authentication (sign-up, login, social auth).

Day 2: Database & Backend Setup

- Choose a database (e.g., PostgreSQL, Supabase, or Firebase).
- Set up models for users, rides, and payments.
- Implement backend API routes (Next.js API routes or a separate Node.js backend).

Day 3: User Dashboard & Profile

- Design and implement the user dashboard (with Tailwind CSS).
- Allow users to update their profile (name, phone number, profile picture).

Day 4: Map & Ride Request System

- Integrate Google Maps API for location selection.
- Implement ride request form (pickup, drop-off, ride type).
- Store ride requests in the database.

Day 5: Driver Dashboard & Ride Matching

- Create a driver dashboard for accepting rides.
- Implement logic to match riders with nearby drivers.

- Display ride status updates (e.g., requested, accepted, in progress).

Day 6: Real-Time Updates & Notifications (Optional)

- Use WebSockets or Firebase Realtime Database for live ride status updates.
- Implement in-app notifications for ride acceptance and status changes.

Day 7: Buffer Day & Testing

- Fix any bugs from previous tasks.
- Write basic unit tests for API routes.
- Ensure authentication and ride booking work properly.

Week 2: Payments, Reviews & Deployment

Day 8: Payment Integration with Stripe

- Set up Stripe for ride fare calculation.
- Implement payment processing for riders.
- Allow drivers to receive payments.

Day 9: Ride History & Receipts

- Implement ride history for users and drivers.
- Generate ride receipts and send via email.

Day 10: Ratings & Reviews

- Allow users to rate drivers and leave reviews.
- Display average ratings on driver profiles.

Day 11: UI Enhancements & Responsiveness

- Improve UI/UX for a smoother experience.
- Ensure mobile responsiveness with Tailwind CSS.

Day 12: Security & Performance Optimization

- Secure API endpoints.
- Optimize database queries and frontend performance.

Day 13: Final Testing & Bug Fixes

- Test the complete workflow (sign-up, booking, payment, ride completion).
- Fix any remaining bugs.

Day 14: Deployment & Launch

- Deploy the application on Vercel (frontend) and a backend hosting service (if separate backend).
- Final demo and documentation.

Use Cases :

1. User Registration & Authentication

- **Use Case:** A new rider registers using email, phone number, or social login (via Clerk).
- **Actors:** Rider
- **Flow:**
    1. Rider enters details (name, email, phone number, password).
    2. System verifies the details and creates an account.
    3. Rider logs in using credentials or social authentication.

---

2. User Profile Management

- **Use Case:** Rider can update their personal details and payment methods.
- **Actors:** Rider
- **Flow:**
    1. Rider navigates to the profile section.
    2. Updates personal info (name, phone number, profile picture).
    3. Adds or removes payment methods.
    4. System saves changes.

---

3. Booking a Ride

- **Use Case:** Rider requests a ride to a destination.
- **Actors:** Rider, System
- **Flow:**
    1. Rider enters the pickup and drop-off locations.
    2. System calculates the estimated fare and shows ride options.
    3. Rider confirms the booking.
    4. System searches for an available driver and assigns one.
    5. Rider gets ride confirmation and estimated arrival time.

---

4. Ride Tracking (Real-Time GPS Tracking)

- **Use Case:** Rider tracks the ride status in real time.
- **Actors:** Rider, System
- **Flow:**
    1. Once the driver is assigned, the rider sees the driver’s live location.
    2. System updates the estimated time of arrival (ETA).
    3. Rider receives notifications (e.g., "Driver is arriving," "Ride started").
    4. During the ride, the app displays real-time route tracking.

---

5. Ride Fare Estimation

- **Use Case:** Rider sees the estimated fare before booking.
- **Actors:** Rider, System
- **Flow:**
    1. Rider selects pickup and destination points.
    2. System calculates the fare based on distance and demand.
    3. Estimated fare is displayed before confirmation.

---

6. Payment Processing via Stripe

- **Use Case:** Rider makes a secure payment for the ride.
- **Actors:** Rider, Payment Gateway (Stripe), System
- **Flow:**
    1. After the ride ends, the system calculates the final fare.
    2. Rider selects a payment method (Card, Wallet, UPI, etc.).
    3. Payment is processed via Stripe.
    4. A receipt is generated and sent to the rider.

---

7. Ride History & Receipts

- **Use Case:** Rider can view past rides and download receipts.
- **Actors:** Rider, System
- **Flow:**
    1. Rider navigates to the "Ride History" section.
    2. System fetches and displays past rides.
    3. Rider can download receipts for payments.

---

8. Cancelling a Ride

- **Use Case:** Rider cancels a ride before pickup.
- **Actors:** Rider, System
- **Flow:**
    1. Rider clicks "Cancel Ride."
    2. System checks if cancellation fees apply (based on time).
    3. If valid, the ride is canceled, and the rider is notified.

---

9. Ratings & Reviews

- **Use Case:** Rider rates and reviews the driver after a ride.
- **Actors:** Rider
- **Flow:**
    1. After the ride, the app prompts the rider to leave a rating (1-5 stars).
    2. Rider can provide optional feedback.
    3. System stores and displays the driver's overall rating.

---

10. Multi-Stop Ride Option

- **Use Case:** Rider adds multiple destinations before booking.
- **Actors:** Rider, System
- **Flow:**
    1. Rider enters multiple stops before confirming the ride.
    2. System calculates the route and fare accordingly.
    3. Driver follows the updated navigation.

---

11. Promo Codes & Discounts

- **Use Case:** Rider applies a discount code before booking.
- **Actors:** Rider, System
- **Flow:**
    1. Rider enters a promo code before confirming the ride.
    2. System validates and applies the discount.
    3. The final fare reflects the discount amount.

---

12. Emergency SOS Feature

- **Use Case:** Rider can trigger an SOS alert during emergencies.
- **Actors:** Rider, System
- **Flow:**
    1. Rider clicks the "SOS" button during a ride.
    2. System shares the rider’s live location with emergency contacts.
    3. System alerts emergency services (if integrated).

---

13. Customer Support & Help Center

- **Use Case:** Rider reports issues or contacts support.
- **Actors:** Rider, Admin
- **Flow:**
    1. Rider navigates to "Help & Support."
    2. Selects an issue category (e.g., payment issue, lost item).
    3. Submits a ticket or contacts customer support.
    4. Admin reviews and responds.

These use cases cover the **core functionalities** of an **Uber Clone**. Let me know if you need more details on any specific feature! 🚀

Resources

Youtube video for the project :
🛵 Ride Booking (Customer + Rider) Full Stack | Node JS | Websocket | Best Project MERN

Build an Uber Clone App with MERN Stack | Complete 11 Hour Masterclass (2025)

Map Integration resources :

1. Leaflet.js Docs (Best for Beginners)

- Leaflet Quick Start Guide : https://leafletjs.com/examples/quick-start/
- Leaflet Routing Machine (for routes) : https://www.liedman.net/leaflet-routing-machine/

2. Mapbox (If you want modern design)

- Mapbox GL JS Docs : https://docs.mapbox.com/mapbox-gl-js/guides/

3. OpenStreetMap (for free map tiles)

- OpenStreetMap Wiki : https://wiki.openstreetmap.org/wiki/Develop

📽️ YouTube Tutorials

Here are high-quality free tutorials you can follow:

Using Leaflet.js (Recommended for MVP)

Leaflet crash course | All you need to know about leaflet | Leaflet | Tekson

1. Leaflet JS Tutorial for Beginners – basics of maps, markers, popups :
2. Leaflet Routing Machine Tutorial – add routing (directions)
3. React + Leaflet Integration – add maps in React apps

Using Mapbox

1. Mapbox GL JS Tutorial – complete guide : Getting Started with Mapbox GL JS Part I

2. React + Mapbox – build maps in React :
   react mapbox gl integration to show marker on map | React Beginners Tutorial

Using Google Maps API (if you want same feel as Google Maps)

1. Google Maps JavaScript API Crash Course

2. React Google Maps Tutorial